

Competitive Programming

Section 40 Editorial: Data Structures Mixed Contest

Competitive Programming Course

2026-08-14

Table of contents

1	Category coverage	4
2	Editorial overview	5
3	A. Hotel Queries	6
3.1	Restatement	6
3.2	Algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	7
4	B. List Removals	9
4.1	Restatement	9
4.2	Algorithm	9
4.3	Correctness proof	9
4.4	Complexity	9
4.5	Full C++ solution	9
4.6	Full Python solution	10
5	C. Mo Distinct Queries	12
5.1	Restatement	12
5.2	Algorithm	12
5.3	Correctness proof	12
5.4	Complexity	12
5.5	Full C++ solution	12
5.6	Full Python solution	13
6	D. Salary Queries	15
6.1	Restatement and constraint implications	15
6.2	Coordinate collection and trace	15
6.3	Algorithm	15
6.4	Correctness proof	16
6.5	Complexity, implementation notes, and tests	16
6.6	Full C++ solution	16
6.7	Full Python solution	18
7	Testing discussion	20
8	Detailed upsolve workshop	21
8.1	A. Hotel Queries	21
8.2	B. List Removals	21
8.3	C. Mo Distinct Queries	22
9	What to carry forward	23

10 E. Functional Walk Minimum	24
10.1 Restatement and constraint recognition	24
10.2 Table meaning	24
10.3 Answering a query	24
10.4 Correctness proof	25
10.5 Complexity and implementation notes	25
10.6 Tests that matter	25
10.7 Full C++ solution	25
10.8 Full Python solution	25
11 F. Running Lower Median	28
11.1 Restatement and invariant choice	28
11.2 Algorithm	28
11.3 Correctness proof	28
11.4 Complexity and common mistakes	29
11.5 Tests that matter	29
11.6 Full C++ solution	29
11.7 Full Python solution	29
12 G. Streaming Room Count	30
12.1 Model and algorithm	30
12.2 Why it works	30
12.3 Full solutions	30
13 H. Toggle K-th Active	32
13.1 Model and algorithm	32
13.2 Why it works	32
13.3 Full solutions	32
14 I. Dynamic Maximum Subarray	34
14.1 Node state	34
14.2 Correctness and complexity	34
14.3 Full solutions	34
15 J. Sliding Median Cost	36
15.1 Reduction	36
15.2 Correctness and complexity	36
15.3 Full solutions	36
16 K. Budget Prefix Search	39
16.1 Segment information and guided descent	39
16.2 Full solutions	39
17 L. Dynamic Inversion Swaps	42
17.1 Local change formula	42
17.2 Data structure, proof, and cost	42
17.3 Full solutions	42

1 Category coverage

Category	Coverage problem
Fenwick tree	B. List Removals; D. Salary Queries
segment tree	A. Hotel Queries
static queries and binary lifting	E. Functional Walk Minimum
heaps and priority queues	F. Running Lower Median

The two additional blind-practice problems are self-contained local packages and are included by this section's checker. C. Mo Distinct Queries remains extra offline-query practice.

2 Editorial overview

Problem	Main idea
A. Hotel Queries	See full algorithm and code below.
B. List Removals	See full algorithm and code below.
C. Mo Distinct Queries	See full algorithm and code below.
D. Salary Queries	Compress update values, then maintain salary frequencies in a Fenwick tree.

3 A. Hotel Queries

3.1 Restatement

For each group size, place it in the first hotel with enough remaining capacity, subtract that size, and print the hotel index or 0.

3.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

3.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

3.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

int n, q;
if (!(cin >> n >> q)) return 0;
int size = 1;
while (size < n) size <= 1;
vector<long long> seg(2 * size, 0);
for (int i = 0; i < n; i++) cin >> seg[size + i];
for (int i = size - 1; i >= 1; i--) seg[i] = max(seg[2 * i], seg[2 * i + 1]);
for (int qi = 0; qi < q; qi++) {
    long long x;
    cin >> x;
    int ans = 0;
    if (seg[1] >= x) {
        int v = 1;
        while (v < size) {
            if (seg[2 * v] >= x) v *= 2;
            else v = 2 * v + 1;
        }
        ans = v - size + 1;
        seg[v] -= x;
        for (v >= 1; v; v >= 1) seg[v] = max(seg[2 * v], seg[2 * v + 1]);
    }
    if (qi) cout << ' ';
    cout << ans;
}
cout << '\n';
return 0;
}

```

3.6 Full Python solution

```

import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    req = data[2 + n:2 + n + q]
    size = 1
    while size < n:
        size *= 2
    seg = [0] * (2 * size)
    for i, x in enumerate(a):
        seg[size + i] = x
    for i in range(size - 1, 0, -1):
        seg[i] = max(seg[2 * i], seg[2 * i + 1])
    out = []
    for x in req:

```

```

if seg[1] < x:
    out.append("0")
    continue
v = 1
while v < size:
    if seg[2 * v] >= x:
        v *= 2
    else:
        v = 2 * v + 1
pos = v - size
seg[v] -= x
v //= 2
while v:
    seg[v] = max(seg[2 * v], seg[2 * v + 1])
    v //= 2
out.append(str(pos + 1))
print(" ".join(out))

if __name__ == "__main__":
    main()

```

4 B. List Removals

4.1 Restatement

Repeatedly remove and print the k -th currently alive element.

4.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

4.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

4.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<int> bit;
    Fenwick(int n) : n(n), bit(n + 1, 0) {}
};
```

```

void add(int i, int delta) {
    for (; i <= n; i += i & -i) bit[i] += delta;
}

int kth(int k) const {
    int pos = 0;
    int step = 1;
    while ((step << 1) <= n) step <<= 1;
    for (; step; step >>= 1) {
        int nxt = pos + step;
        if (nxt <= n && bit[nxt] < k) {
            pos = nxt;
            k -= bit[nxt];
        }
    }
    return pos + 1;
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    Fenwick bit(n);
    for (int i = 1; i <= n; i++) bit.add(i, 1);
    for (int t = 0; t < n; t++) {
        int k;
        cin >> k;
        int idx = bit.kth(k);
        if (t) cout << ' ';
        cout << a[idx];
        bit.add(idx, -1);
    }
    cout << '\n';
    return 0;
}

```

4.6 Full Python solution

```

import sys

class Fenwick:
    def __init__(self, n):
        self.n = n
        self.bit = [0] * (n + 1)
    def add(self, i, delta):

```

```

        while i <= self.n:
            self.bit[i] += delta
            i += i & -i
def kth(self, k):
    pos = 0
    bit = 1 << self.n.bit_length()
    while bit:
        nxt = pos + bit
        if nxt <= self.n and self.bit[nxt] < k:
            pos = nxt
            k -= self.bit[nxt]
        bit //= 2
    return pos + 1

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    a = data[1:1 + n]
    positions = data[1 + n:1 + 2 * n]
    bit = Fenwick(n)
    for i in range(1, n + 1):
        bit.add(i, 1)
    out = []
    for k in positions:
        idx = bit.kth(k)
        out.append(str(a[idx - 1]))
        bit.add(idx, -1)
    print(" ".join(out))

if __name__ == "__main__":
    main()

```

5 C. Mo Distinct Queries

5.1 Restatement

For each offline range query, print the number of distinct values in the range.

5.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

5.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

5.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

5.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Query {
    int l, r, id;
};
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<int> a(n), vals;
    for (int &x : a) {
        cin >> x;
        vals.push_back(x);
    }
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    for (int &x : a) x = int(lower_bound(vals.begin(), vals.end(), x) - vals.begin());
    vector<Query> queries(q);
    for (int i = 0; i < q; i++) {
        cin >> queries[i].l >> queries[i].r;
        --queries[i].l; --queries[i].r;
        queries[i].id = i;
    }
    int block = max(1, (int)sqrt(n));
    sort(queries.begin(), queries.end(), [&](const Query& A, const Query& B) {
        int ba = A.l / block, bb = B.l / block;
        if (ba != bb) return ba < bb;
        return (ba & 1) ? A.r > B.r : A.r < B.r;
    });
    vector<int> freq(vals.size(), 0), ans(q);
    int distinct = 0, l = 0, r = -1;
    auto add = [&](int pos) {
        if (freq[a[pos]]++ == 0) distinct++;
    };
    auto remove = [&](int pos) {
        if (--freq[a[pos]] == 0) distinct--;
    };
    for (auto qu : queries) {
        while (r < qu.r) add(++r);
        while (r > qu.r) remove(r--);
        while (l < qu.l) remove(l++);
        while (l > qu.l) add(--l);
        ans[qu.id] = distinct;
    }
    for (int x : ans) cout << x << '\n';
    return 0;
}

```

5.6 Full Python solution

```

import sys
from collections import defaultdict

```

```

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    idx = 2 + n
    block = max(1, int(n ** 0.5))
    queries = []
    for qi in range(q):
        l, r = data[idx] - 1, data[idx + 1] - 1
        idx += 2
        queries.append((l, r, qi))
    queries.sort(key=lambda x: (x[0] // block, x[1] if (x[0] // block) % 2 == 0 else -x[1]))
    freq = defaultdict(int)
    ans = [0] * q
    distinct = 0
    left, right = 0, -1
    for l, r, qi in queries:
        while right < r:
            right += 1
            freq[a[right]] += 1
            if freq[a[right]] == 1:
                distinct += 1
        while right > r:
            freq[a[right]] -= 1
            if freq[a[right]] == 0:
                distinct -= 1
            right -= 1
        while left < l:
            freq[a[left]] -= 1
            if freq[a[left]] == 0:
                distinct -= 1
            left += 1
        while left > l:
            left -= 1
            freq[a[left]] += 1
            if freq[a[left]] == 1:
                distinct += 1
        ans[qi] = distinct
    print("\n".join(map(str, ans)))

if __name__ == "__main__":
    main()

```

6 D. Salary Queries

6.1 Restatement and constraint implications

There are n employees with current integer salaries. An operation $! k x$ changes employee k 's salary to x . A query $? a b$ asks how many current salaries lie in the inclusive *value* interval $[a, b]$. There are up to 200,000 employees and operations, and salaries can be negative or as large as 10^9 in magnitude.

The interval is about salary values, not employee indices. Scanning all employees per query is quadratic. A query needs only the number of occurrences of each ordered salary value, which is exactly what a Fenwick tree can store after coordinate compression.

6.2 Coordinate collection and trace

Read and retain all operations first. Collect every initial salary and every new x in an update, sort the distinct values, and give each one a one-based rank. This preserves order: for collected salaries u and v , $u < v$ if and only if $\text{rank}(u) < \text{rank}(v)$.

For initial salaries $[10, 20, 30, 40, 50]$, query $? 15 45$ has four salaries at most 45 and one salary below 15, so its answer is $4 - 1 = 3$. After $! 2 35$, remove one count at rank 20 and add one at rank 35. Then $? 30 35$ returns two. The bounds 15 and 45 need not be coordinates: binary search locates where they fall among actual salary values.

6.3 Algorithm

```
collect initial salaries and every update value; sort and deduplicate them
insert all current salary ranks into a Fenwick frequency tree

for each operation:
    if ! k x:
        remove the rank of current[k]
        set current[k] = x and add its rank
    else ? a b:
        right = number of coordinate values <= b
        left = number of coordinate values < a
        print BIT.prefix(right) - BIT.prefix(left)
```

Use `upper_bound(coords, b) / bisect_right(coords, b)` for `right`, and `lower_bound(coords, a) / bisect_left(coords, a)` for `left`.

6.4 Correctness proof

The Fenwick invariant is: the frequency at each rank equals the number of employees currently earning that corresponding salary. Initialization adds each employee once. An update removes exactly its old salary and adds exactly its new one, so the invariant remains true.

For $[a, b]$, `right` covers exactly salaries at most b , while `left` covers exactly salaries below a . The two Fenwick prefix sums therefore count those two employee sets under the invariant. Their difference counts exactly the employees with $a \leq \text{salary} \leq b$, so every query answer is correct. QED.

6.5 Complexity, implementation notes, and tests

Sorting coordinates costs $O((n+q) \log(n+q))$. Each update/query uses a constant number of binary searches and Fenwick operations, hence $O(\log(n+q))$; memory is $O(n+q)$. Use `long long` for C++ salary values.

Test an update below every old salary, equal salaries, query endpoints absent from coordinates, and an empty value range. Common mistakes are using a lower bound for the inclusive upper endpoint, or adding the new salary without first removing the old one.

6.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    vector<int> bit;
    explicit Fenwick(int n) : bit(n + 1, 0) {}

    void add(int index, int delta) {
        for (int n = (int)bit.size(); index < n; index += index & -index) {
            bit[index] += delta;
        }
    }

    int sum(int index) const {
        int result = 0;
        for (; index > 0; index -= index & -index) result += bit[index];
        return result;
    }
};

struct Operation {
    char type;
    int first;
    long long second;
};
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<long long> salary(n + 1), coords;
    for (int i = 1; i <= n; i++) {
        cin >> salary[i];
        coords.push_back(salary[i]);
    }

    vector<Operation> operations;
    operations.reserve(q);
    for (int i = 0; i < q; i++) {
        char type;
        long long first, second;
        cin >> type >> first >> second;
        operations.push_back({type, (int)first, second});
        if (type == '!') coords.push_back(second);
    }

    sort(coords.begin(), coords.end());
    coords.erase(unique(coords.begin(), coords.end()), coords.end());
    auto rank_of = [&](long long value) {
        return int(lower_bound(coords.begin(), coords.end(), value) - coords.begin()) + 1;
    };

    Fenwick bit((int)coords.size());
    for (int i = 1; i <= n; i++) bit.add(rank_of(salary[i]), 1);

    for (const Operation& operation : operations) {
        if (operation.type == '!') {
            int employee = operation.first;
            bit.add(rank_of(salary[employee]), -1);
            salary[employee] = operation.second;
            bit.add(rank_of(salary[employee]), 1);
        } else {
            long long low = operation.first;
            long long high = operation.second;
            int before_low = int(lower_bound(coords.begin(), coords.end(), low) - coords.begin());
            int through_high = int(upper_bound(coords.begin(), coords.end(), high) - coords.begin());
            cout << bit.sum(through_high) - bit.sum(before_low) << '\n';
        }
    }
    return 0;
}

```

6.7 Full Python solution

```
import bisect
import sys

class Fenwick:
    def __init__(self, size: int) -> None:
        self.bit = [0] * (size + 1)

    def add(self, index: int, delta: int) -> None:
        while index < len(self.bit):
            self.bit[index] += delta
            index += index & -index

    def prefix_sum(self, index: int) -> int:
        total = 0
        while index:
            total += self.bit[index]
            index -= index & -index
        return total

def main() -> None:
    tokens = sys.stdin.buffer.read().split()
    if not tokens:
        return
    pointer = 0
    n = int(tokens[pointer])
    q = int(tokens[pointer + 1])
    pointer += 2
    salary = [0] + [int(value) for value in tokens[pointer : pointer + n]]
    pointer += n

    operations: list[tuple[bytes, int, int]] = []
    coords = salary[1:]
    for _ in range(q):
        kind = tokens[pointer]
        first = int(tokens[pointer + 1])
        second = int(tokens[pointer + 2])
        pointer += 3
        operations.append((kind, first, second))
        if kind == b"!":
            coords.append(second)

    coords = sorted(set(coords))
    bit = Fenwick(len(coords))

    def rank_of(value: int) -> int:
        return bisect.bisect_left(coords, value) + 1
```

```

for value in salary[1:]:
    bit.add(rank_of(value), 1)

output: list[str] = []
for kind, first, second in operations:
    if kind == b"!":
        employee, new_salary = first, second
        bit.add(rank_of(salary[employee]), -1)
        salary[employee] = new_salary
        bit.add(rank_of(new_salary), 1)
    else:
        low, high = first, second
        before_low = bisect.bisect_left(coords, low)
        through_high = bisect.bisect_right(coords, high)
        output.append(str(bit.prefix_sum(through_high) - bit.prefix_sum(before_low)))
print("\n".join(output))

if __name__ == "__main__":
    main()

```

7 Testing discussion

The small D cases keep a literal salary list and answer queries by scanning it; they never invoke a reference solution. The scale case combines 100,000 updates with 100,000 range-count queries, so a linear scan cannot pass. The expected outputs are shared by the C++ and Python runners.

8 Detailed upsolve workshop

8.1 A. Hotel Queries

Constraints and trap. n, q are up to about $2e5$. A linear scan for each group is too slow; a plain maximum alone finds feasibility but not the first index. Store the maximum remaining capacity in each segment-tree node.

Trace and pseudocode. With capacities $[3, 1, 4, 1, 5]$, a request of four descends root $\rightarrow$ left/right according to whether the left subtree maximum is at least four, reaches index three, and changes it to zero.

```
if root.max < x: output 0
otherwise descend: go left if left.max >= x, else right
at leaf i: subtract x; recompute max values on the path to root; output i
```

Proof and complexity. Every node is the maximum of exactly its leaves. A feasible left child contains an earlier valid hotel; an infeasible left child contains none, so the descent returns the first valid leaf. Each query changes and rebuilds one root-to-leaf path: $O(\log n)$ time, $O(n)$ memory. Use long long capacities. Test no feasible hotel, the first/last feasible hotel, and repeated requests to one hotel.

8.2 B. List Removals

Constraints and trap. Removing an element from an array/vector shifts all following values and becomes quadratic. Instead, original positions are fixed; a Fenwick tree stores one for alive positions and zero for removed positions.

Trace and pseudocode. For values $[10, 20, 30, 40, 50]$, rank two resolves to original index two, outputs 20, then adds -1. With counts $[1, 0, 1, 1, 1]$, rank three resolves to original index four, outputting 40.

```
initialize BIT with one at every index
for rank k:
    i = first index whose BIT prefix sum >= k
    output value[i]; BIT.add(i, -1)
```

Proof and complexity. The BIT prefix sum is the number of alive original positions at or before its index. Therefore its first prefix reaching k is the k -th current item. Fenwick binary lifting gives $O(\log n)$ per removal and $O(n)$ memory. The search uses one-based indices; test rank one, last rank, and duplicate values. A small mutable-list simulation is the stress oracle.

8.3 C. Mo Distinct Queries

Constraints and trap. Static array, many offline range queries, and no updates signal Mo's algorithm. Recomputing a set for every range is quadratic in the worst case. Coordinate-compress values before using an array frequency.

Trace and pseudocode. For $[1,2,1,3,2,4]$, activate $[1,3]$ to obtain frequencies $1:2, 2:1$ and answer two. Move to $[2,5]$: remove the first one, add 3 and 2, and obtain three.

```
sort queries by left block, alternating right direction
move [curL, curR] one endpoint at a time
add(pos): if freq[a[pos]] changes 0->1, distinct++
remove(pos): if freq[a[pos]] changes 1->0, distinct--
```

Proof and complexity. The frequency invariant exactly represents the current window, so `distinct` is correct when a query is reached. Block order limits endpoint movement to $O((n+q)\sqrt{n})$ in the standard analysis; memory is $O(n+q)$. Convert endpoints once, use buffered I/O in Python, and test all equal values, all distinct values, singletons, and repeated ranges against `len(set(...))` on small random arrays.

9 What to carry forward

Choose a stored summary that answers the decision you actually need: subtree maximum for first-fit, alive prefix count for dynamic rank, and frequencies for an offline window. The invariant makes both the data structure and its proof mechanical.

The fixed samples cover visible behavior. The random tests generate many small operation sequences and compute expected outputs from the reference Python implementation, then the same expected files are used against both the C++ and Python submissions.

10 E. Functional Walk Minimum

10.1 Restatement and constraint recognition

Every vertex has exactly one outgoing edge. A query starts at v , follows exactly k edges, and asks for both the final vertex and the minimum value among the start and all reached vertices.

Simulating a query takes $O(k)$, but k may be as large as 10^{18} . The functional graph never branches during a walk, so binary lifting can precompute the effect of blocks of $1, 2, 4, 8, \dots$ steps. Alongside the destination of a block, store an aggregate—in this case, a minimum.

10.2 Table meaning

For vertex v and level j , define:

```
up[v][j] = vertex reached after exactly  $2^j$  edges from  $v$ 
mn[v][j] = minimum value among the  $2^j$  reached vertices
           (the starting value at  $v$  is not included)
```

Excluding the block's starting value avoids double-counting concerns when concatenating blocks. The query initializes its answer with `value[v]`, so the original start is still included exactly as required.

For level zero:

```
up[v][0] = next[v]
mn[v][0] = value[next[v]]
```

Two adjacent blocks of length $2^{(j-1)}$ give:

```
middle    = up[v][j-1]
up[v][j]   = up[middle][j-1]
mn[v][j]   = min(mn[v][j-1], mn[middle][j-1])
```

10.3 Answering a query

Write k in binary. Starting with `answer = value[v]`, inspect its set bits from low to high. For bit j , merge `mn[v][j]` into the answer and replace v by `up[v][j]`. The selected blocks concatenate to exactly k edges. Sixty-one levels cover every $k \leq 10^{18}$.

10.4 Correctness proof

By induction on j , $up[v][j]$ reaches the endpoint of exactly 2^j edges and $mn[v][j]$ is the minimum over precisely the vertices reached by those edges. The base case is the single outgoing edge. For the induction step, the first stored half starts at v ; the second starts at its endpoint `middle`. Their concatenation is the required full block, and taking the minimum of their aggregates is exact.

During a query, after processing some bits, v is the endpoint of the corresponding concatenated blocks and `answer` is the minimum over the original start and all vertices reached so far. Processing a set bit appends the correct next block by the table lemma and merges exactly its values. After all bits, the blocks total k , proving both reported values correct.

10.5 Complexity and implementation notes

With $L = 61$, preprocessing costs $O(nL)$ time and memory, and each query costs $O(L)$. In asymptotic terms this is $O((n+q) \log K)$ time and $O(n \log K)$ memory.

The Python reference stores tables in packed `array` objects rather than lists of Python integers; at $n = 200000$, ordinary nested lists would consume far more memory. Be careful that $k = 0$ must return the original vertex and its value, and that the block minimum includes destinations but not its start.

10.6 Tests that matter

<code>k = 0</code>	-> unchanged vertex and starting value
<code>k = 1</code>	-> one table base block
<code>k</code> with several set bits	-> block concatenation
A self-loop	-> repeated endpoint
A tail entering a cycle	-> works without cycle detection
Minimum occurs at the start, in the tail, and inside the cycle	
<code>k = 10¹⁸</code>	-> verifies the number of levels

For small graphs and small k , direct step-by-step simulation is an independent oracle.

10.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; using ll=long long;
int main(){ios::sync_with_stdio(false); cin.tie(nullptr); int n,q; cin>>n>>q; const int L=61; vec
```

10.8 Full Python solution

```

import sys
from array import array

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, q = data[:2]
    next_vertex = array("i", (x - 1 for x in data[2 : 2 + n]))
    value = data[2 + n : 2 + 2 * n]
    index = 2 + 2 * n

    levels = 61
    up = [next_vertex]
    minimum = [array("q", (value[next_vertex[v]] for v in range(n)))]

    for level in range(1, levels):
        previous_up = up[level - 1]
        previous_minimum = minimum[level - 1]
        up.append(array("i", (previous_up[previous_up[v]] for v in range(n))))
        minimum.append(
            array(
                "q",
                (
                    min(
                        previous_minimum[v],
                        previous_minimum[previous_up[v]],
                    )
                    for v in range(n)
                ),
            )
        )

    output = []
    for _ in range(q):
        vertex = data[index] - 1
        steps = data[index + 1]
        index += 2
        answer = value[vertex]
        bit = 0

        while steps:
            if steps & 1:
                answer = min(answer, minimum[bit][vertex])
                vertex = up[bit][vertex]
            steps //= 2
            bit += 1

        output.append(f"{vertex + 1} {answer}")

    sys.stdout.write("\n".join(output))

```

```
if __name__ == "__main__":  
    main()
```

11 F. Running Lower Median

11.1 Restatement and invariant choice

After each insertion, output the element at sorted position $(\text{length}+1)//2$: the lower of the two middle values when the length is even. Sorting the entire prefix after every insertion would take $O(n^2 \log n)$.

Split the values into two heaps:

```
lower = max-heap containing the smaller half
upper = min-heap containing the larger half
```

Maintain two invariants:

1. every value in **lower** is at most every value in **upper**;
2. $\text{len}(\text{lower})$ equals $\text{len}(\text{upper})$ or is exactly one larger.

Under these invariants, **lower.top()** is always the lower median.

11.2 Algorithm

For each arriving value **x**:

1. If **lower** is empty or $x \leq \text{lower.top}()$, insert it into **lower**; otherwise insert it into **upper**.
2. If **lower** is smaller than **upper**, move **upper.top()** to **lower**.
3. If **lower** has more than one extra value, move **lower.top()** to **upper**.
4. Output **lower.top()**.

Python's `heapq` is a min-heap, so store negated values in **lower**.

11.3 Correctness proof

Initially both invariants hold. The insertion comparison places **x** on the correct side of the current boundary, preserving the ordering invariant. Rebalancing moves only the maximum of the lower side or the minimum of the upper side across that boundary, so ordering remains true while the size invariant is restored.

If the total count is odd, **lower** contains one more element than **upper**; its maximum is the unique middle element. If the count is even, the heaps have equal size; the maximum of **lower** is the smaller middle element. Thus the reported top is exactly the requested lower median after every insertion.

11.4 Complexity and common mistakes

Each value causes one insertion and at most one heap transfer, all $O(\log n)$. Total time is $O(n \log n)$ and memory is $O(n)$.

- Equal values may go into either side, but the ordering and size invariants must still hold.
- Returning the upper heap's top for even prefixes gives the upper median.
- Rebalancing sizes without preserving the partition order is insufficient.
- In Python, remember to negate both when pushing to and reading from the lower max-heap.

11.5 Tests that matter

One value.
Strictly increasing and strictly decreasing streams.
All values equal.
Alternating very small and very large values.
Negative values.
Even prefixes where lower and upper medians differ.

A simple oracle sorts each short prefix and reads index $(\text{len}(\text{prefix})-1)//2$.

11.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;cin>>n;priority_queue<long long>
```

11.7 Full Python solution

```
import heapq,sys
def main():
    d=list(map(int,sys.stdin.buffer.read().split()));a=d[1:];lo=[];hi=[];out=[]
    for x in a:
        if not lo or x<=-lo[0]:heapq.heappush(lo,-x)
        else:heapq.heappush(hi,x)
        if len(lo)<len(hi):heapq.heappush(lo,-heapq.heappop(hi))
        if len(lo)>len(hi)+1:heapq.heappush(hi,-heapq.heappop(lo))
        out.append(str(-lo[0]))
    print(" ".join(out))
if __name__=="__main__":main()
```

12 G. Streaming Room Count

12.1 Model and algorithm

Because meetings arrive by starting time, keep the ending times of precisely the meetings active at the next start in a min-heap. Before inserting $[s, e)$, remove every ending time at most s . Insert e and update the largest heap size ever seen.

12.2 Why it works

After the removals, every heap entry overlaps the new meeting, and every previous meeting not in the heap has ended. Thus the heap size after insertion is exactly the simultaneous load at s . The minimum room count is the maximum simultaneous load over all starts, which the running maximum records.

Time is $O(n \log n)$ and memory is $O(n)$. The important boundary is that an end equal to a start is reusable because intervals are half-open.

12.3 Full solutions

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n))
        return 0;
    priority_queue<long long, vector<long long>, greater<long long>> ends;
    int answer = 0;
    for (int i = 0; i < n; i++) {
        long long s, e;
        cin >> s >> e;
        while (!ends.empty() && ends.top() <= s)
            ends.pop();
        ends.push(e);
        answer = max(answer, (int)ends.size());
        if (i)
            cout << ' ';
        cout << answer;
    }
}
```

```
cout << '\n';  
}
```

```
import heapq, sys  
def main():  
    data=list(map(int,sys.stdin.buffer.read().split()))  
    if not data:return  
    n=data[0]; ends=[]; answer=0; out=[]; j=1  
    for _ in range(n):  
        s,e=data[j:j+2]; j+=2  
        while ends and ends[0]<=s: heapq.heappop(ends)  
        heapq.heappush(ends,e); answer=max(answer,len(ends)); out.append(str(answer))  
    print(*out)  
if __name__=="__main__":main()
```

13 H. Toggle K-th Active

13.1 Model and algorithm

Store 1 for an active position and 0 otherwise in a Fenwick tree. Toggling is a point addition. A K k query asks for the smallest index whose prefix sum is at least k ; find it by Fenwick binary lifting after checking that the total count reaches k .

13.2 Why it works

Every Fenwick prefix is exactly the number of active positions in that prefix. Therefore the first prefix reaching k ends at the k -th active position. Binary lifting maintains the largest prefix whose count is still below k , so the following index is the answer.

Each operation costs $O(\log n)$ and memory is $O(n)$. Test repeated toggles, $k=1$, $k=\text{total}$, and an invalid k .

13.3 Full solutions

```
#include <bits/stdc++.h>
using namespace std;
struct BIT {
    int n;
    vector<int> t;
    BIT(int n) : n(n), t(n + 1) {}
    void add(int i, int v) {
        for (; i <= n; i += i & -i)
            t[i] += v;
    }
    int sum(int i) {
        int s = 0;
        for (; i; i -= i & -i)
            s += t[i];
        return s;
    }
    int kth(int k) {
        int p = 0;
        for (int d = bit_floor((unsigned)n); d; d >>= 1)
            if (p + d <= n && t[p + d] < k)
                k -= t[p += d];
        return p + 1;
    }
}
```

```

};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q))
        return 0;
    BIT b(n);
    vector<char> on(n + 1);
    while (q--) {
        char c;
        int x;
        cin >> c >> x;
        if (c == 'T') {
            b.add(x, on[x] ? -1 : 1);
            on[x] ^= 1;
        } else
            cout << (b.sum(n) < x ? -1 : b.kth(x)) << '\n';
    }
}

```

```

import sys
def main():
    data=sys.stdin.buffer.read().split()
    if not data:return
    n,q=map(int,data[:2]); bit=[0]*(n+1); on=[False]*(n+1); out=[]; j=2
    def add(i,v):
        while i<=n: bit[i]+=v;i+=i&-i
    def total():
        i=n;s=0
        while i:s+=bit[i];i-=i&-i
        return s
    def kth(k):
        p=0;d=1<<(n.bit_length()-1)
        while d:
            np=p+d
            if np<=n and bit[np]<k:p=np;k-=bit[np]
            d//=2
        return p+1
    for _ in range(q):
        c=data[j];x=int(data[j+1]);j+=2
        if c==b'T':add(x,-1 if on[x] else 1);on[x]=not on[x]
        else:out.append(str(kth(x) if total()>=x else -1))
    print('\n'.join(out))
if __name__=="__main__":main()

```

14 I. Dynamic Maximum Subarray

14.1 Node state

For each segment store its total sum, best nonempty prefix, best nonempty suffix, and best nonempty subarray. When joining left L and right R , a best subarray is either inside L , inside R , or $L.\text{suffix} + R.\text{prefix}$. Prefix and suffix have the analogous two choices.

14.2 Correctness and complexity

Those cases exhaust where a contiguous interval can cross the unique boundary between the children. Leaves are correct for a one-element segment, so the merge proves every node correct by induction. A point update changes one root path: $O(\log n)$ time per update and $O(n)$ memory. Use negative infinity for padding; using zero would incorrectly permit an empty subarray.

14.3 Full solutions

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct N {
    ll sum, pref, suff, best;
};
N one(ll x) { return {x, x, x, x}; }
N join(N a, N b) {
    return {a.sum + b.sum, max(a.pref, a.sum + b.pref),
            max(b.suff, b.sum + a.suff), max({a.best, b.best, a.suff + b.pref})};
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q))
        return 0;
    int z = 1;
    while (z < n)
        z *= 2;
    const ll X = -(1LL << 60);
    vector<N> t(2 * z, {0, X, X, X});
    for (int i = 0; i < n; i++) {
        ll x;
```

```

    cin >> x;
    t[z + i] = one(x);
}
for (int i = z - 1; i; i--)
    t[i] = join(t[2 * i], t[2 * i + 1]);
while (q--) {
    int p;
    ll x;
    cin >> p >> x;
    p = z + p - 1;
    t[p] = one(x);
    for (p /= 2; p; p /= 2)
        t[p] = join(t[2 * p], t[2 * p + 1]);
    cout << t[1].best << '\n';
}
}

```

```

import sys
def main():
    d=list(map(int,sys.stdin.buffer.read().split()))
    if not d:return
    n,q=d[:2];z=1
    while z<n:z*=2
    neg=-10**30;t=[(0,neg,neg,neg)]*(2*z)
    def one(x):return (x,x,x,x)
    def join(a,b):
        return (a[0]+b[0], max(a[1],a[0]+b[1]),
                max(b[2],b[0]+a[2]), max(a[3],b[3],a[2]+b[1]))
    for i,x in enumerate(d[2:2+n]):t[z+i]=one(x)
    for i in range(z-1,0,-1):t[i]=join(t[2*i],t[2*i+1])
    j=2+n;out=[]
    for _ in range(q):
        p,x=d[j:j+2];j+=2;p=z+p-1;t[p]=one(x);p//=2
        while p:t[p]=join(t[2*p],t[2*p+1]);p//=2
        out.append(str(t[1][3]))
    print('\n'.join(out))
if __name__=="__main__":main()

```

15 J. Sliding Median Cost

15.1 Reduction

The minimizing x is any median. Coordinate-compress all values and maintain two Fenwick trees for the current window: one stores counts and the other stores sums. Find the lower median m by order statistics. If (cL, sL) are the count and sum at most m , and (cR, sR) are those above it, the cost is

$$m \cdot cL - sL + sR - m \cdot cR.$$

15.2 Correctness and complexity

Moving x toward a median cannot increase the number of values moved away from compared with values moved toward; hence a median minimizes absolute deviation. The formula separately sums $m - a[i]$ on the left and $a[i] - m$ on the right. Each insertion, deletion, and median search is $O(\log n)$, for $O(n \log n)$ total time and $O(n)$ memory.

15.3 Full solutions

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct BIT {
    int n;
    vector<ll> t;
    BIT(int n) : n(n), t(n + 1) {}
    void add(int i, ll v) {
        for (; i <= n; i += i & -i)
            t[i] += v;
    }
    ll sum(int i) {
        ll s = 0;
        for (; i; i -= i & -i)
            s += t[i];
        return s;
    }
    int kth(ll k) {
        int p = 0;
        for (int d = bit_floor((unsigned)n); d; d >>= 1)
            if (p + d <= n && t[p + d] < k)
```

```

        k -= t[p += d];
    return p + 1;
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    if (!(cin >> n >> k))
        return 0;
    vector<ll> a(n), v;
    for (ll &x : a)
        cin >> x;
    v = a;
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());
    BIT cnt(v.size()), sum(v.size());
    auto add = [&](ll x, int d) {
        int p = lower_bound(v.begin(), v.end(), x) - v.begin() + 1;
        cnt.add(p, d);
        sum.add(p, d * x);
    };
    for (int i = 0; i < k; i++)
        add(a[i], 1);
    for (int l = 0; l++) {
        int p = cnt.kth((k + 1) / 2);
        ll m = v[p - 1], cl = cnt.sum(p), sl = sum.sum(p), ct = cnt.sum(v.size()),
            st = sum.sum(v.size());
        ll ans = m * cl - sl + (st - sl) - m * (ct - cl);
        if (l)
            cout << ' ';
        cout << ans;
        if (l + k == n)
            break;
        add(a[l], -1);
        add(a[l + k], 1);
    }
    cout << '\n';
}

```

```

import bisect,sys
def main():
    d=list(map(int,sys.stdin.buffer.read().split()))
    if not d:return
    n,k=d[:2];a=d[2:];v=sorted(set(a));z=len(v);bc=[0]*(z+1);bs=[0]*(z+1)
    def add(bit,i,x):
        while i<=z:bit[i]+=x;i+=i&-i
    def pref(bit,i):
        s=0
        while i:s+=bit[i];i-=i&-i
        return s

```

```

def change(x,d):
    p=bisect.bisect_left(v,x)+1;add(bc,p,d);add(bs,p,d*x)
def kth(kth):
    p=0;step=1<<(z.bit_length()-1)
    while step:
        np=p+step
        if np<=z and bc[np]<kth:p=np;kth-=bc[np]
        step//=2
    return p+1
for x in a[:k]:change(x,1)
out=[]
for l in range(n-k+1):
    p=kth((k+1)//2);m=v[p-1];cl=pref(bc,p);sl=pref(bs,p);ct=pref(bc,z);st=pref(bs,z)
    out.append(str(m*cl-sl+st-sl-m*(ct-cl)))
    if l+k<n:change(a[l],-1);change(a[l+k],1)
print(*out)
if __name__=="__main__":main()

```

16 K. Budget Prefix Search

16.1 Segment information and guided descent

Signed values destroy monotonicity: a running sum may cross the budget, fall back, and cross again. Store for every segment its total sum and its maximum nonempty prefix sum. The merge is

```
sum = left.sum + right.sum
pref = max(left.pref, left.sum + right.pref).
```

During a query, maintain the sum of the already skipped part of $[1, n]$. A whole segment can be skipped exactly when `running + segment.pref <= budget`; then add its total to `running`. Otherwise descend left before right. The first leaf that cannot be skipped is the first budget breach.

The stored maximum examines every prefix of a segment: it either stays in the left child or contains all of the left child and a prefix of the right. Thus a skipped node contains no breach. Descending left first returns the earliest node that does contain one. Point assignments and searches both cost $O(\log n)$, with $O(n)$ memory.

16.2 Full solutions

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Node {
    ll sum, pref;
};
struct Seg {
    int n;
    vector<Node> t;
    Seg(vector<ll> &a) : n(a.size()), t(4 * n) { build(1, 0, n, a); }
    void pull(int x) {
        t[x] = {t[2 * x].sum + t[2 * x + 1].sum,
                max(t[2 * x].pref, t[2 * x].sum + t[2 * x + 1].pref)};
    }
    void build(int x, int l, int r, vector<ll> &a) {
        if (r - l == 1) {
            t[x] = {a[l], a[l]};
            return;
        }
        int m = (l + r) / 2;
```

```

    build(2 * x, l, m, a);
    build(2 * x + 1, m, r, a);
    pull(x);
}

void set_value(int x, int l, int r, int p, ll v) {
    if (r - l == 1) {
        t[x] = {v, v};
        return;
    }
    int m = (l + r) / 2;
    if (p < m)
        set_value(2 * x, l, m, p, v);
    else
        set_value(2 * x + 1, m, r, p, v);
    pull(x);
}

int first(int x, int l, int r, int ql, ll budget, ll &sum) {
    if (r <= ql)
        return -1;
    if (ql <= l && sum + t[x].pref <= budget) {
        sum += t[x].sum;
        return -1;
    }
    if (r - l == 1)
        return l;
    int m = (l + r) / 2, res = first(2 * x, l, m, ql, budget, sum);
    return res < 0 ? first(2 * x + 1, m, r, ql, budget, sum) : res;
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q))
        return 0;
    vector<ll> a(n);
    for (ll &x : a)
        cin >> x;
    Seg s(a);
    while (q--) {
        char c;
        int i;
        ll x;
        cin >> c >> i >> x;
        --i;
        if (c == 'U')
            s.set_value(1, 0, n, i, x);
        else {
            ll sum = 0;
            int answer = s.first(1, 0, n, i, x, sum);
            cout << (answer < 0 ? -1 : answer + 1) << '\n';
        }
    }
}

```

```

    }
}
}

```

```

import sys
def main():
    d=sys.stdin.buffer.read().split()
    if not d:return
    n,q=map(int,d[:2]);a=list(map(int,d[2:2+n]));total=[0]*(4*n);pref=[0]*(4*n)
    def pull(x):total[x]=total[2*x]+total[2*x+1];pref[x]=max(pref[2*x],total[2*x]+pref[2*x+1])
    def build(x,l,r):
        if r-l==1:total[x]=pref[x]=a[l];return
        m=(l+r)//2;build(2*x,l,m);build(2*x+1,m,r);pull(x)
    def update(x,l,r,p,v):
        if r-l==1:total[x]=pref[x]=v;return
        m=(l+r)//2
        if p<m:update(2*x,l,m,p,v)
        else:update(2*x+1,m,r,p,v)
        pull(x)
    def first(x,l,r,ql,budget,running):
        if r<=ql:return -1,running
        if ql<=l and running+pref[x]<=budget:return -1,running+total[x]
        if r-l==1:return l,running
        m=(l+r)//2;answer,running=first(2*x,l,m,ql,budget,running)
        return (answer,running) if answer>=0 else first(2*x+1,m,r,ql,budget,running)
    build(1,0,n);j=2+n;out=[]
    for _ in range(q):
        c=d[j];i=int(d[j+1])-1;x=int(d[j+2]);j+=3
        if c==b'U':update(1,0,n,i,x)
        else:answer,_=first(1,0,n,i,x,0);out.append(str(answer+1 if answer>=0 else -1))
    print('\n'.join(out))
if __name__=="__main__":main()

```

17 L. Dynamic Inversion Swaps

17.1 Local change formula

Assume $i < j$, $x = a[i]$, and $y = a[j]$. Pairs wholly outside these positions do not change. If $x < y$, their mutual pair contributes +1 after swapping. Every middle value z with $x < z < y$ changes two non-inversions into two inversions; all other middle values have zero net change. Therefore

```
delta = +(1 + 2*count(i < k < j and x < a[k] < y)).
```

For $x > y$, the same quantity is subtracted.

17.2 Data structure, proof, and cost

Split positions into blocks and keep each block sorted. Count middle values in the open value interval using binary search on full blocks and direct checks on the two boundary fragments. After a swap, rebuild its one or two blocks. The formula considers every pair involving a changed position exactly once, so adding it preserves the exact inversion total.

Compute the initial total with a Fenwick tree. With block size about $\sqrt{n}$, a swap costs $O(\sqrt{n} \log n)$, initialization costs $O(n \log n)$, and memory is $O(n)$. Open value bounds and $i=j$ are essential edge cases.

17.3 Full solutions

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q))
        return 0;
    vector<int> a(n);
    for (int &x : a)
        cin >> x;
    vector<int> bit(n + 1);
    auto add = [&](int i) {
        for (; i <= n; i += i & -i)
            bit[i]++;
    };
```

```

};
auto sum = [&](int i) {
    int s = 0;
    for (; i; i -= i & -i)
        s += bit[i];
    return s;
};
ll inv = 0;
for (int x : a) {
    inv += sum(n) - sum(x);
    add(x);
}
int B = 450, nb = (n + B - 1) / B;
vector<vector<int>> blk(nb);
auto rebuild = [&](int b) {
    blk[b].assign(a.begin() + b * B, a.begin() + min(n, (b + 1) * B));
    sort(blk[b].begin(), blk[b].end());
};
for (int b = 0; b < nb; b++)
    rebuild(b);
auto count = [&](int l, int r, int lo, int hi) {
    int ans = 0;
    if (l > r || lo >= hi)
        return 0;
    while (l <= r && l % B) {
        ans += lo < a[l] && a[l] < hi;
        l++;
    }
    while (l + B - 1 <= r) {
        auto &v = blk[l / B];
        ans += lower_bound(v.begin(), v.end(), hi) -
            upper_bound(v.begin(), v.end(), lo);
        l += B;
    }
    while (l <= r) {
        ans += lo < a[l] && a[l] < hi;
        l++;
    }
    return ans;
};
while (q--) {
    int i, j;
    cin >> i >> j;
    --i;
    --j;
    if (i > j)
        swap(i, j);
    if (i != j) {
        int x = a[i], y = a[j], c = count(i + 1, j - 1, min(x, y), max(x, y));
        ll d = 1 + 2LL * c;
        inv += (x < y ? d : -d);
    }
}

```

```

        swap(a[i], a[j]);
        rebuild(i / B);
        if (i / B != j / B)
            rebuild(j / B);
    }
    cout << inv << '\n';
}
}

```

```

import bisect,math,sys
def main():
    d=list(map(int,sys.stdin.buffer.read().split()))
    if not d:return
    n,q=d[:2];a=d[2:2+n];bit=[0]*(n+1);inv=0
    def pref(i):
        s=0
        while i:s+=bit[i];i-=i&-i
        return s
    for seen,x in enumerate(a):
        inv+=seen-pref(x)
        i=x
        while i<=n:bit[i]+=1;i+=i&-i
    B=max(1,int(math.sqrt(n))+1);nb=(n+B-1)//B;blocks=[]
    for b in range(nb):blocks.append(sorted(a[b*B:min(n,(b+1)*B)]))
    def rebuild(b):blocks[b]=sorted(a[b*B:min(n,(b+1)*B)])
    def count(l,r,lo,hi):
        ans=0
        while l<=r and l%B:ans+=lo<a[l]<hi;l+=1
        while l+B-1<=r:
            v=blocks[l//B];ans+=bisect.bisect_left(v,hi)-bisect.bisect_right(v,lo);l+=B
        while l<=r:ans+=lo<a[l]<hi;l+=1
        return ans
    out=[];p=2+n
    for _ in range(q):
        i,j=d[p]-1,d[p+1]-1;p+=2
        if i>j:i,j=j,i
        if i!=j:
            x,y=a[i],a[j];delta=1+2*count(i+1,j-1,min(x,y),max(x,y));inv+=delta if x<y else
            a[i],a[j]=y,x;rebuild(i//B)
            if i//B!=j//B:rebuild(j//B)
        out.append(str(inv))
    print('\n'.join(out))
if __name__=="__main__":main()

```